Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM302 Үйлдлийн систем

Бие даалт №1. Banker's Algorithm manual

Шалгасан багш: Г.Ганбат /F.CS10/
Гүйцэтгэсэн оюутан: Ц.Цэндсүрэн /B242270128/
/B231940023/
/B241910004/
/B241940034/
/B242270041/

Улаанбаатар хот
2026 он

Хэсэг 1. Хэрэглэгчийн заавар

1.1 Програмын зорилго

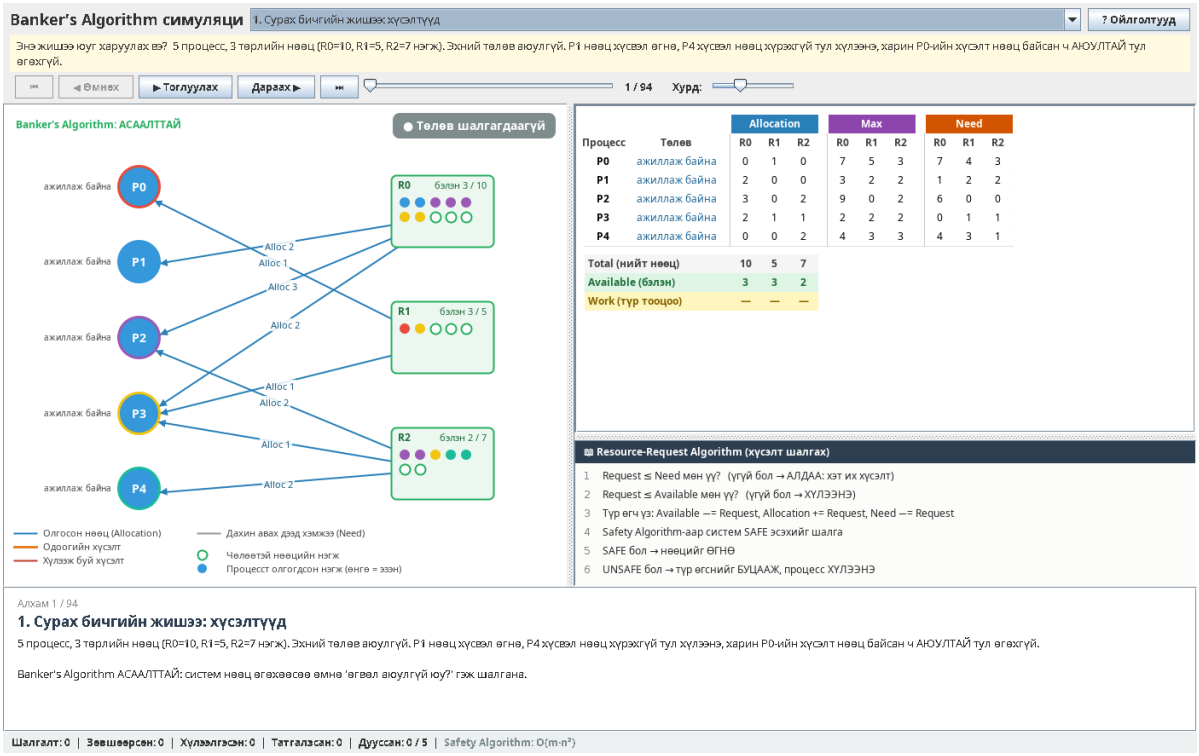
Энэ програм нь **Banker's Algorithm**-ийг (түгжрэлээс сэргийлэх алгоритм) алхам алхмаар харуулдаг симулятор юм. Процессууд нөөц хүсэхэд систем 'олгосны дараа бүх процесс эцэст нь ажлаа дуусгаж чадах уу?' гэж шалгаж, аюулгүй бол л олгоно. Та процесс ба нөөцийн холбоог *Resource-allocation graph*-аар, тоон өгөгдлийг матрицаар, алгоритмын одоогийн мөрийг псевдокодоор нэгэн зэрэг харна.

1.2 Дэмжих орчин

Зүйл	Шаардлага
Үйлдлийн систем	Windows 10/11, Ubuntu 22.04+/24.04, macOS (график орчинтой)
Java	JDK 17 эсвэл түүнээс шинэ (`javac -version`). Туршсан: OpenJDK 21
Дэлгэц	1100×720 px-ээс их (анхны хэмжээ 1400×880). Монгол кирилл дэмждэг фонт (Windows: Segoe UI/Arial; Linux: DejaVu/Noto)
Сүлжээ	Ажиллуулахад шаардлагагүй (зөвхөн `git clone`-д)

1.3 Суулгах ба нээх алхам

- **1.** JDK суулгасан эсэхээ шалгана: терминал/Command Prompt нээгээд `java -version` ба `javac -version`.
- **2.** Кодыг татна: `git clone https://github.com/Tsendsvren/csm302_biydaalt.git`, дараа нь `cd csm302_biydaalt`.
- **3.** Compile: Linux/macOS дээр `./build.sh`, Windows дээр `build.bat`.
- **4.** Ажиллуулах: `./run.sh` эсвэл `run.bat`. 'II-4-13: Banker's Algorithm' цонх нээгдэнэ.



Зураг 1. Програмыг нээсэн анхны байдал (туршилт 1, алхам 1/94).

1.4 Оролт ба тохиргоо

Одоогийн хувилбарт оролтын өгөгдөл нь **10 бэлэн туршилт** (дээд талын жагсаалтаас сонгоно). туршилт бүр дараахыг тодорхойлно: Available, Max, Allocation, хүсэлтүүдийн дараалал, Banker's Algorithm асаалттай эсэх.

№	туршилт	Banker	Үр дүн
1	Сурах бичгийн жишээ: хүсэлтүүд	Асаалттай	P1 олгоно, P4 хүлээнэ, P0 татгалзана → DONE
2	Нөөц байгаа ч өгөхгүй	Асаалттай	Татгалзсан 3 → DONE (түгжрэлгүй)
3	Банкиргүй: DEADLOCK	Унтраалттай	DEADLOCK
4	Deadlock илрүүлэлт: түгжрэл ҮГҮЙ	Унтраалттай	DONE
5	Deadlock илрүүлэлт: түгжрэл БАЙНА	Унтраалттай	DEADLOCK
6	4 нөөц, дасгал 8.3	Асаалттай	DONE
7	Дасгал 8.9 (а): Available=(0,3,0,1)	Асаалттай	DEADLOCK (эхний төлөв UNSAFE)
8	Дасгал 8.9 (б): Available=(1,0,0,2)	Асаалттай	DONE
9	Дасгал 8.28: хүсэлтүүд	Асаалттай	DONE
10	Том систем: 10 процесс, 4 нөөц	Асаалттай	DONE

1.5 Ажиллуулах, зогсоох, reset хийх

Удирдлага	Үйлдэл
Дээд талын жагсаалт	туршилт сонгоно. Сонгох бүрд симуляци эхнээс нь дахин тооцоологдож, 1-р алхмаас эхэлнэ (reset)
|◀	Эхний алхам руу буцна (reset-тэй адил, туршилт солихгүй)
◀ Өмнөх / Дараах ▶	Нэг алхам хойш / урагш. Автомат тоглуулалтыг зогсооно
▶|	Сүүлийн алхам (эцсийн дүн) руу харайна
▶ Тоглуулах / Зогсоох	Алхмуудыг автоматаар урагш нь тоглуулна; дахин дарвал зогсоно. Төгсгөлд хүрвэл өөрөө зогсоно. Төгсгөлд байхад дарвал эхнээс нь тоглуулна
Seek slider	Дурын алхам руу шууд харайна; баруун талд 'одоогийн / нийт' алхам
Хурд slider	Автомат тоглуулалтын алхам хоорондын хугацаа (баруун тийш = хурдан)
? Ойлголтууд	Max, Need, Allocation, Available, SAFE/UNSAFE/DEADLOCK-ийн товч тайлбар
Цонх хаах	Програм дуусна (`EXIT_ON_CLOSE`)

1.6 Дүрслэлийн тэмдэг, өнгө, төлөв

Resource-allocation graph (зүүн панел)

Тэмдэг	Утга
Дугуй P0, P1, … (зүүн)	Процесс. Дотор нь өнгө = төлөв, хүрээний өнгө = процессын өвөрмөц өнгө. Шар гэрэлтэй = одоогийн алхамд оролцож буй процесс
Ногоон хайрцаг R0, R1, … (баруун)	Нөөцийн төрөл. Дээр нь 'бэлэн X / нийт Y'. Дотор нэгж бүр нэг цэг
Өнгөт дүүрэн цэг	Процесст олгогдсон нөөцийн нэгж. Өнгө нь эзэн процессын хүрээний өнгөтэй таарна
Цагаан (ногоон хүрээтэй) цэг	Чөлөөтэй нөөцийн нэгж
Цэнхэр сум (нөөц → процесс), 'Alloc k'	Олгосон нөөц (Allocation)
Бүдүүн улбар шар сум (процесс → нөөц), 'хүсэлт k'	Одоогийн хүсэлт
Улаан сум, 'хүлээж k'	Хүлээж буй (одоохондоо олгогдоогүй) хүсэлт
Саарал сум, 'Need k'	Тухайн процессын дахин авах дээд хэмжээ (идэвхтэй процессод)
Баруун дээд badge	• SAFE (ногоон) · • UNSAFE (улбар шар) · • DEADLOCK (улаан) · • Төлөв шалгагдаагүй (саарал)
Зүүн дээд бичиг	Banker's Algorithm: АСААЛТТАЙ (ногоон) / УНТРААТАЙ (улаан)

Процессын төлөв (граф дахь дугуйн өнгө ба матрицын мөрийн өнгө)

Төлөв	Дугуйн өнгө	Матрицын мөр	Утга
ажиллаж байна	цэнхэр	цагаан	Энгийн, юу ч хүсээгүй
хүсэж байна	улбар шар	цайвар улбар шар	Нөөц хүсч байна, систем шалгаж байна
шалгаж байна	шар	цайвар шар	Safety Algorithm энэ процессыг шалгаж байна
хүлээж байна	ягаан	цайвар ягаан	Хүсэлт нь одоохондоо олгогдоогүй
дууссан	ногоон (бүдэг)	цайвар ногоон	Ажлаа дуусгаж нөөцөө буцаасан
DEADLOCK	улаан	цайвар улаан	Түгжрэлд орсон

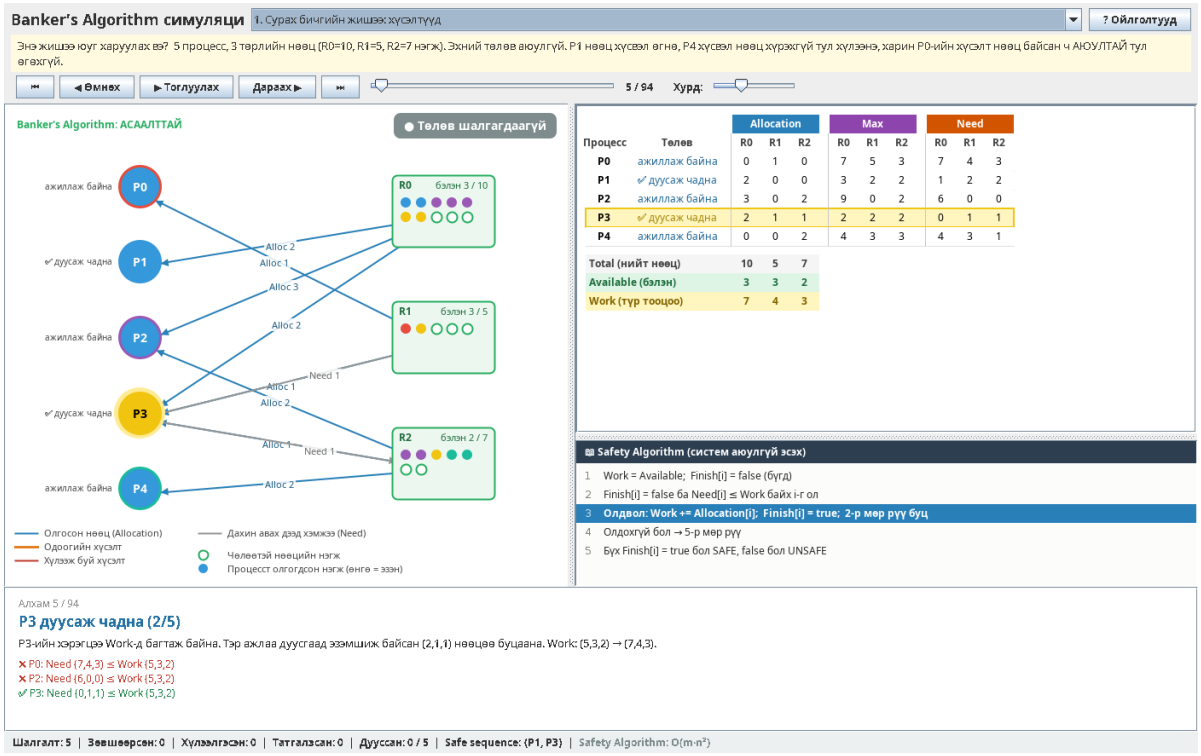
Матриц, псевдокод, тайлбар, тоолуур

- **Матриц (баруун дээд):** Allocation (цэнхэр), Max (нил ягаан), Need (улбар шар) гурван бүлэг; доор нь Total, Available, Work (түр тооцоо), Request (одоогийн хүсэлт). Хүсэлт гаргаж буй процессын Need-ийн өөрчлөгдөх нүд улбар шараар тодорно.
- **Псевдокод (баруун доод):** Resource-Request, Safety Algorithm, Deadlock Detection гэсэн гурван блок. Тухайн алхамд харгалзах мөр цэнхэр дэвсгэртэй.
- **Тайлбар (доод):** алхмын гарчиг ба тайлбар; ✔ шалгалт тэнцсэн, ✘ тэнцээгүй. Өнгө: ногоон = амжилт, улаан = татгалзсан/алдаа/түгжрэл, улбар шар = хүсэлт, цэнхэр = шалгалт.
- **Тоолуур (хамгийн доод):** Шалгалт · Зөвшөөрсөн · Хүлээлгэсэн · Татгалзсан · Дууссан X/N · Safe sequence.

1.7 Жишээнүүд

Жишээ 1. 'Нөөц байгаа ч АЮУЛТАЙ тул олгохгүй' (туршилт 1)

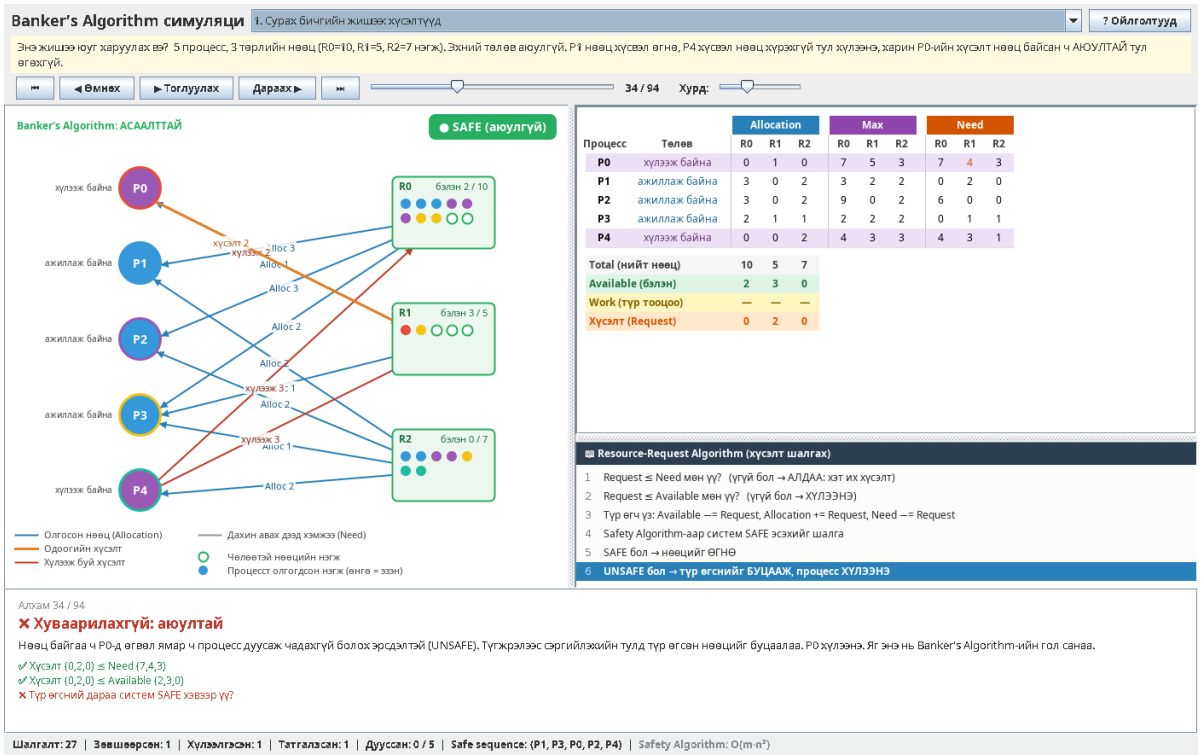
- Жагсаалтаас **1. Сурах бичгийн жишээ: хүсэлтүүд**-ийг сонгоно (анхнаасаа сонгогдсон).
- ▶ Дараах товчийг дарж Safety Algorithm-ийг ажиглана: Work (3,3,2) → P1 дуусч чадна → (5,3,2) → P3 → (7,4,3) … Эцэст нь badge 'SAFE' болно.
- Алхам ∼34 руу (seek slider-аар) очно: P0 (0,2,0) хүсэлт. Хоёр шалгалт ✔, харин 'түр өгсний дараа SAFE хэвээр үү?' ✘. Псевдокодын 6-р мөр тодорч, P0 хүлээнэ.
- ▶| дарж төгсгөл рүү очно: бүх 5 процесс дууссан, 'Татгалзсан: 2'.



Зураг 2. P0-ийн хүсэлт татгалзагдсан алхам: ✘ 'түр өгсний дараа систем SAFE хэвээр үү?'.

Жишээ 2. Banker-тэй ба Banker-гүй харьцуулалт (туршилт 2 ба 3)

- **2. Нөөц байгаа ч өгөхгүй** Banker АСААЛТТАЙ-ийг сонгож төгсгөл рүү очно: бүх процесс дуусна, түгжрэл гарахгүй.
- Дараа нь **3. Банкиргүй бол: DEADLOCK**-ийг сонгоно. Яг ижил систем боловч нөөц байгаа тул P2-д шууд өгөх ба төгсгөлд badge **DEADLOCK**, P0 ба P2 улаан болно.



Зураг 3. Banker унтраалттай үед түгжрэл: чөлөөтэй 4 нэгж, харин P0-д 5, P2-д 6 хэрэгтэй.

Жишээ 3 (нэмэлт). Ижил хүснэгт, өөр Available (туршилт 7 ба 8) туршилт 7 (Available=(0,3,0,1)) эхний төлөв UNSAFE, 8 (Available=(1,0,0,2)) SAFE. Хоёулангийн эхний Safety Algorithm-ийг ажиглаж, зөвхөн Available-ийн өөрчлөлт дүгнэлтийг эсрэгээр болгосныг харна.

1.8 Түгээмэл алдаа ба шийдэл

Шинж тэмдэг	Шалтгаан	Шийдэл
`'javac' is not recognized` / `javac: command not found`	Зөвхөн JRE суусан, JDK биш; эсвэл PATH-д алга	JDK 17+ суулгана (Temurin/OpenJDK). Терминалаа дахин нээж `javac -version` шалгана
`UnsupportedClassVersionError`	Compile хийсэн Java-аас хуучин Java-аар ажиллуулсан	`java -version` ба `javac -version` ижил болгоно; `out/`-ийг устгаад дахин build
`Could not find or load main class com.os.Main`	Build хийгээгүй эсвэл буруу хавтсаас ажиллуулсан	Репозиторийн үндсэн хавтсаас build дараа нь run; `out/com/os/Main.class` байгаа эсэхийг шалгана
Монгол үсэг □□□ болж харагдана	Кирилл дэмжсэн фонт байхгүй (голдуу Linux)	`sudo apt install fonts-dejavu fonts-noto-core`
Эмодзи/✔ тэмдэгт дөрвөлжин	Системийн фонт тэр тэмдэгтгүй	Үйл ажиллагаанд нөлөөгүй, дүрслэлийн асуудал. Noto фонт суулгана
Цонх хэт жижиг/том, хэсэг нь таслагдсан	Дэлгэцийн scale (HiDPI)	Цонхыг томруулна (min 1100×720); эсвэл `java -Dsun.java2d.uiScale=1 -cp out com.os.Main`
Linux дээр `HeadlessException`	DISPLAY байхгүй (SSH, сервер)	График орчинд ажиллуулна; ганцаараа тест хийх бол `./test.sh` (GUI хэрэггүй)
Windows-д `./build.sh` ажиллахгүй	Bash биш	`build.bat`, `run.bat` ашиглана

Хэсэг 2. Operational заавар

2.1 Repository clone хийх

```
git clone https://github.com/Tsendsvren/csm302_biydaalt.git
cd csm302_biydaalt
git log --oneline    # 4 commit, үндсэн branch: main
```

2.2 OS, compiler/runtime, version

Зүйл	Шаардлага	Баталгаажуулсан орчин
OS	Windows 10+, Linux, macOS	Ubuntu 24.04
JDK (javac + java)	17 буюу түүнээс шинэ (`record` ашигласан)	OpenJDK 21.0.12 (javac 21.0.12.1)
Build tool	Шаардлагагүй (Maven/Gradle ашиглаагүй)	`javac`
GUI-ийн орчин	Swing дэмждэг график session	Xvfb (виртуал дэлгэц)
Git	Clone хийхэд	—

Нээлттэй баталгаагүй зүйл: Windows/macOS дээр энэ заавраар build/run хийж туршаагүй (скриптүүд `.bat` болон `.sh` хоёулаа бичигдсэн). Шүүмжлэгч баг эхний ажиллуулалтаар шалгана уу.

2.3 Dependency суулгах

Гадаад dependency байхгүй. Зөвхөн JDK-ийн стандарт сан (`javax.swing`, `java.awt`). Юу ч татаж суулгах шаардлагагүй.

2.4 Build ба Run команд

	Linux / macOS	Windows (cmd)
Build	`./build.sh`	`build.bat`
Run (GUI)	`./run.sh`	`run.bat`
Автомат тест	`./test.sh`	`test.bat`

Скрипт ашиглахгүй бол гараар:

```
# Linux / macOS
mkdir -p out
javac -encoding UTF-8 -d out src/main/java/com/os/*.java
java -Dfile.encoding=UTF-8 -cp out com.os.Main

# Windows (cmd)
mkdir out
javac -encoding UTF-8 -d out src\main\java\com\os\*.java
java -Dfile.encoding=UTF-8 -cp out com.os.Main
```

Хүлээгдсэн: build дуусахад `out/com/os/*.class` үүснэ, алдаа/warning гарахгүй; run хийхэд

1400×880 цонх нээгдэнэ (Зураг 1).

2.5 Үндсэн тестийн өгөгдөл ба хүлээгдсэн үр дүн

`./test.sh` нь build хийж, `tests/AlgorithmTests.java`-г ажиллуулна. GUI шаардахгүй. Гаралтын төгсгөл:

```
=== ДҮН: 29 PASS, 0 FAIL ===   (exit code 0; FAIL байвал exit code 1)
```

Өгөгдөл (сурах бичгийн жишээ)	Хүлээгдсэн үр дүн
Available=(3,3,2); Max, Allocation = туршилт 1	SAFE. Дараалал ⟨P1,P3,P0,P2,P4⟩ (сурах бичгийн ⟨P1,P3,P4,P0,P2⟩-тэй өөр боловч хоёулаа хүчинтэй)
P1 хүснэ (1,0,2)	GRANTED; Available → (2,3,0)
Дараа нь P4 хүснэ (3,3,0)	MUST_WAIT
Дараа нь P0 хүснэ (0,2,0)	UNSAFE → татгалзана, төлөв өөрчлөгдөхгүй
туршилт 2 / 3 эцсийн алхам	DONE / DEADLOCK
туршилт 4 / 5 эцсийн алхам	DONE / DEADLOCK

2.6 Нэмэлт boundary / error тест

Тест	Ролт	Хүлээгдсэн
T5	P1 хүснэ (9,0,0) > Need (1,2,2)	EXCEEDS_NEED
T6a	Процессын ID = 9 (хүрээнээс гадуур)	INVALID
T6b	Хүсэлт (−1,0,0)	INVALID
T6c	Буруу урттай вектор (2 утга, 3 нөөц)	INVALID
T7a	Allocation > Max	`IllegalArgumentException`: 'P0, R0: Allocation нь Max-аас их байж болохгүй.'
T7b	Available < 0	`IllegalArgumentException`: 'Available[0] сөрөг байж болохгүй.'
T8a / T8b	Тэг хүсэлт; 1 процесс, Need=0	GRANTED; SAFE
E1	Хүсэлт Need-ээс их (туршилтйн хөдөлгүүрт)	ERROR алхам, 'Татгалзсан' тоолуур 1
E2	Бүх Need = 0	DONE, 2/2
E3 / E4	Эхнээсээ түгжрэлтэй; Max > нийт нөөц	DEADLOCK
F1–F3	3000 санамсаргүй төлөв (seed=42)	Брут-форстой зөрүү 0; Safe sequence бүр хүчинтэй; зөвшөөрөгдсөний дараа SAFE

Өөрийн өгөгдлөөр турших (код засаж). `ScenarioLibrary.all()`-ийн `List.of(...)`-д дараахыг нэмээд дахин build хийнэ:

```
new Scenario("11. Миний туршилт", "Тайлбар текст", true,
new int[]{2},                  // Available
```

```
            new int[][]{{3}, {3}},          // Max (процесс бүрээр)
            new int[][]{{1}, {1}},          // Allocation
            List.of(req(0, 1)))             // хүсэлтүүд: P0 нэг нэгж
```

Буруу өгөгдөл (жишээ нь Allocation > Max) өгвөл тухайн туршилтйг сонгох үед (эхний туршилт бол програм эхлэхэд) `SystemState` нь `IllegalArgumentException` шиднэ. Алдааны мессеж одоогоор дэлгэцэнд биш, зөвхөн консолд (stderr) гарна.

2.7 Known issues

- UI дээрээс туршилт/параметр засварлах, гараар хүсэлт оруулах боломж байхгүй (`ScenarioLibrary`-г кодоор засна).
- Хөдөлгүүр (`SimulationEngine`) нь `BankerAlgorithm.requestResources`-ийг дууддаггүй, ижил логикийг өөрөө давхар хэрэгжүүлсэн. Fuzz тест зөвхөн `BankerAlgorithm`-ийг шалгасан.
- Deadlock Detection нь Request-ийг Need-тэй тэнцүү гэж авдаг (хамгийн муу тохиолдол).
- Нөөц чөлөөлөх (release) тусдаа хүсэлт, олон thread байхгүй.
- 1400 px өргөнд дээд талын 'Banker's Algorithm симуляци' гарчиг таслагдаж харагдана.
- Windows/macOS дээр туршаагүй; GUI-ийн автомат тест байхгүй.
- Эмодзи ба ✔ тэмдэгтийн харагдац системийн фонтоос хамаарна.